Regresión_Logística(Drugs)

October 12, 2026

1 Actividad: Regresión logística aplicada a la base de datos Drugs

1.1 Materia: Grandes Bases de Datos

1.1.1 Nombre: Ana Camila López Ceballos.

[92]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report
from sklearn import metrics
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

import os
```

[93]:
```
col_names = ["Age","Sex","BP","Cholesterol","Na_to_K","Drug"]
pima = pd.read_csv("drugs.csv", header=0, names=col_names)
df = pima.iloc[1:,:]
df
```

[93]:

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	drugY
5	22	F	NORMAL	HIGH	8.607	drugX
..	…	..	…	…	…	…
195	56	F	LOW	HIGH	11.567	drugC
196	16	M	LOW	HIGH	12.006	drugC
197	52	M	NORMAL	HIGH	9.894	drugX
198	23	M	NORMAL	NORMAL	14.020	drugX
199	40	F	LOW	NORMAL	11.349	drugX

[199 rows x 6 columns]

[94]:
```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])
df["BP"] = le.fit_transform(df["BP"])
df["Cholesterol"] = le.fit_transform(df["Cholesterol"])

df["Drug"] = df["Drug"].map({"drugA": 0, "drugB": 0,"drugC": 0,"drugX":␣
↪1,"drugY": 1})
```

La base de datos contiene variables cualitativas que deben transformarse a valores numéricos para poder utilizarlas en el modelo de regresión logística.

Para ello se utilizó LabelEncoder, donde convertimos las variables `Sex`, `BP` y `Cholesterol` a una escala numérica.

En la variable sex las categorías quedaron codificadas como: F=0 y M=1

En la variable BP las características quedaron como: HIGH=0, LOW=1, NORMAL=2

En la variable Cholesterol la codificación fue: HIGH=0, NORMAL=1

Además, la variable llamada `Drug` fue transformada en una variable binaria de acuerdo a el tipo de proveedor del medicamento, el 0 representa el medicamento de origen nacional y el 1 representa ell medicamento de origen extranjero.

[95]:
```
df
```

[95]:

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
1	47	1	1	0	13.093	0
2	47	1	1	0	10.114	0
3	28	0	2	0	7.798	1
4	61	0	1	0	18.043	1
5	22	0	2	0	8.607	1
..	…	…	..	…	…	…
195	56	0	1	0	11.567	0
196	16	1	1	0	12.006	0
197	52	1	2	0	9.894	1
198	23	1	2	1	14.020	1
199	40	0	1	1	11.349	1

[199 rows x 6 columns]

[96]:
```
#Selección de las características
feature_cols =["Age","Sex","BP","Cholesterol","Na_to_K"]
X = df[feature_cols]
y = df.Drug
```

[97]:
```
X_train, X_test, y_train, y_test =train_test_split(X,y,test_size = 0.
↪2,random_state =1)
```

[98]:
```
#sag: Stochastic Average Gradient
#newton-cg: Library for large linear classification (Algoritmo Coordinante␣
 ↪Descent)
#saga: Variante del método sag
#lbfgs: Limited-memory Broyned_Fletcher_Goldfarb_Shanno Algoritm
```

2 Regresión logística con Solucionador (Solver): sag

[99]:
```
model= LogisticRegression(solver ="sag")
clf = model.fit(X_train, y_train)
```

[100]:
```
#Predicción de etiquetas de clase sobre datos de prueba
y_pred = model.predict (X_test)

#Impresión de coeficientes de la regresión de puntajes
print ("Intercepto (Beta 0)", clf.intercept_)
print("Pesos de cada variable /Beta1,Beta2,...,Beta5)", clf.coef_)
```

```
Intercepto (Beta 0) [-0.3375]
Pesos de cada variable /Beta1,Beta2,…,Beta5) [[-0.0383 -0.1292  0.9911  0.2349
0.1651]]
```

[101]:
```
#Evaluación de la presición del modelo
score = model.score(X_test,y_test)
print("Score de precisión:",score)
```

```
Score de precisión: 0.825
```

[102]:
```
#Reporte de clasificación
print(classification_report(y_test,y_pred))
```

```
              precision    recall  f1-score   support

           0       0.75      0.33      0.46         9
           1       0.83      0.97      0.90        31

    accuracy                           0.82        40
   macro avg       0.79      0.65      0.68        40
weighted avg       0.81      0.82      0.80        40
```

2.0.1 Interpretación:

El modelo con SAG obtuvo una precisión global de 0.825, lo que significa que clasificó correctamente el 82.5% de los casos. En general, el resultado es aceptable, pero no es completamente bueno porque el desempeño cambia mucho entre las dos clases.

Para la clase 0, correspondiente a medicamentos nacionales, la precisión fue de 0.75, el recall de 0.33 y el F1-score de 0.46. Esto indica que el modelo tiene dificultades para identificar correctamente los medicamentos nacionales.

Para la clase 1, correspondiente a medicamentos extranjeros, obtuvo una precisión de 0.83, recall de 0.97 y F1-score de 0.90, por lo que su desempeño en esta clase es mucho mejor.

El modelo con SAG presenta una precisión global aceptable, pero tiene dificultades importantes para identificar los medicamentos nacionales.

3 Regresión logística con Solucionador (Solver): newton-cg

[103]:
```
import numpy as np
np.set_printoptions(precision=4, suppress=True)
```

[104]:
```
model = LogisticRegression(solver="newton-cg")
clf = model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercepto (Beta0):",clf.intercept_)
print("Pesos (Betas):",clf.coef_)
print("------------------------------------------------------------")
score=model.score(X_test, y_test)
print("Precición global:", score)
print("------------------------------------------------------------")
print(classification_report(y_test,y_pred))
```

```
Intercepto (Beta0): [-11.9093]
Pesos (Betas): [[0.0115 0.2503 3.2437 2.0861 0.6666]]
------------------------------------------------------------
Precición global: 0.95
------------------------------------------------------------
              precision    recall  f1-score   support

           0       0.89      0.89      0.89         9
           1       0.97      0.97      0.97        31

    accuracy                           0.95        40
   macro avg       0.93      0.93      0.93        40
weighted avg       0.95      0.95      0.95        40
```

3.0.1 Interpretación:

El modelo con Newton-CG obtuvo una precisión global de 0.95, lo que significa que clasificó correctamente el 95% de los casos. En general, el resultado es bueno porque presenta un desempeño alto y bastante equilibrado entre las dos clases.

Para la clase 0, correspondiente a medicamentos nacionales, la precisión fue de 0.89, el recall de 0.89 y el F1-score de 0.89. Esto indica que el modelo identifica correctamente la mayoría de los medicamentos nacionales y comete pocos errores en esta categoría.

Para la clase 1, correspondiente a medicamentos extranjeros, obtuvo una precisión de 0.97, recall de 0.97 y F1-score de 0.97, por lo que su desempeño en esta clase es excelente.

El modelo con Newton-CG presenta un desempeño muy bueno y más equilibrado que SAG, ya que identifica correctamente tanto los medicamentos nacionales como los extranjeros.

4 Regresión logística con Solucionador (Solver): liblinear

[105]:
```
model = LogisticRegression(solver="liblinear")
clf = model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercepto (Beta0):",clf.intercept_)
print("Pesos (Betas):",clf.coef_)
print("------------------------------------------------------------")
score=model.score(X_test, y_test)
print("Precición global:", score)
print("------------------------------------------------------------")
print(classification_report(y_test,y_pred))
```

```
Intercepto (Beta0): [-3.4745]
Pesos (Betas): [[-0.0261 -0.2196  1.961   1.1363  0.2752]]
------------------------------------------------------------
Precición global: 0.975
------------------------------------------------------------
              precision    recall  f1-score   support

           0       1.00      0.89      0.94         9
           1       0.97      1.00      0.98        31

    accuracy                           0.97        40
   macro avg       0.98      0.94      0.96        40
weighted avg       0.98      0.97      0.97        40
```

4.0.1 Interpretación:

El modelo con Liblinear obtuvo una precisión global de 0.975, lo que significa que clasificó correctamente el 97.5% de los casos. Este resultado es muy bueno y, hasta este punto, es superior al obtenido con SAG y Newton-CG.

Para la clase 0, correspondiente a medicamentos nacionales, la precisión fue de 1.00, el recall de 0.89 y el F1-score de 0.94. Esto indica que todos los casos que el modelo clasificó como nacionales realmente lo eran, aunque no logró identificar el 100% de los nacionales presentes.

Para la clase 1, correspondiente a medicamentos extranjeros, obtuvo una precisión de 0.97, recall de 1.00 y F1-score de 0.98. Esto significa que identificó correctamente todos los medicamentos extranjeros del conjunto de prueba y presentó muy pocos errores.

El modelo con Liblinear presenta un desempeño excelente y bastante equilibrado entre ambas clases. Además, su precisión global de 97.5% es la más alta de los solucionadores evaluados hasta ahora, por lo que es un candidato para ser seleccionado como el modelo final.

5 Regresión logística con Solucionador (Solver): saga

[106]:
```
model = LogisticRegression(solver="saga")
clf = model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercepto (Beta0):",clf.intercept_)
print("Pesos (Betas):",clf.coef_)
print("------------------------------------------------------------")
score=model.score(X_test, y_test)
print("Precición global:", score)
print("------------------------------------------------------------")
print(classification_report(y_test,y_pred))
```

```
Intercepto (Beta0): [-0.1608]
Pesos (Betas): [[-0.0348 -0.0771  0.6363  0.1308  0.1611]]
------------------------------------------------------------
Precición global: 0.825
------------------------------------------------------------
              precision    recall  f1-score   support

           0       0.75      0.33      0.46         9
           1       0.83      0.97      0.90        31

    accuracy                           0.82        40
   macro avg       0.79      0.65      0.68        40
weighted avg       0.81      0.82      0.80        40
```

5.0.1 Interpretación:

El modelo con SAGA obtuvo una precisión global de 0.825, lo que significa que clasificó correctamente el 82.5% de los casos. El resultado es aceptable, pero no es de los mejores porque el desempeño es muy diferente entre las dos clases.

Para la clase 0, correspondiente a medicamentos nacionales, la precisión fue de 0.75, el recall de 0.33 y el F1-score de 0.46. Esto indica que el modelo tiene dificultades para identificar correctamente los medicamentos nacionales.

Para la clase 1, correspondiente a medicamentos extranjeros, obtuvo una precisión de 0.83, recall de 0.97 y F1-score de 0.90, por lo que su desempeño en esta clase es mucho mejor.

SAGA presenta un comportamiento muy parecido a SAG. Aunque identifica bien los medicamentos extranjeros, tiene un rendimiento bajo para los nacionales, por lo que no sería la mejor opción frente a los solucionadores pasados como Newton-CG o Liblinear.

6 Regresión logística con Solucionador (Solver): lbfgs

[107]:
```
model = LogisticRegression(solver="lbfgs")
clf = model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Intercepto (Beta0):",clf.intercept_)
print("Pesos (Betas):",clf.coef_)
print("------------------------------------------------------------")
score=model.score(X_test, y_test)
print("Precición global:", score)
print("------------------------------------------------------------")
print(classification_report(y_test,y_pred))
```

```
Intercepto (Beta0): [-11.9532]
Pesos (Betas): [[0.0116 0.2539 3.2506 2.0924 0.6687]]
------------------------------------------------------------
Precición global: 0.95
------------------------------------------------------------
              precision    recall  f1-score   support

           0       0.89      0.89      0.89         9
           1       0.97      0.97      0.97        31

    accuracy                           0.95        40
   macro avg       0.93      0.93      0.93        40
weighted avg       0.95      0.95      0.95        40
```

6.0.1 Interpretación:

El modelo con LBFGS obtuvo una precisión global de 0.95, lo que significa que clasificó correctamente el 95% de los casos. Este resultado es muy bueno y muestra un desempeño bastante equilibrado entre las dos clases.

Para la clase 0, correspondiente a medicamentos nacionales, la precisión fue de 0.89, el recall de 0.89 y el F1-score de 0.89. Esto indica que el modelo identifica correctamente la mayoría de los medicamentos nacionales.

Para la clase 1, correspondiente a medicamentos extranjeros, obtuvo una precisión de 0.97, recall de 0.97 y F1-score de 0.97, por lo que su desempeño en esta clase es excelente.

LBFGS presenta un desempeño muy bueno y equilibrado, similar al obtenido con Newton-CG. Sin embargo, su precisión global de 95% es ligeramente menor que la obtenida con Liblinear, que alcanzó 97.5%.

[108]:
```
model = LogisticRegression(solver="liblinear")
clf = model.fit(X_train, y_train)
```

[109]:
```
# Predicción de probabilidades para casos particuleres
y_pred_probs = clf.predict_proba(X_test)[:,1]
y_pred_probs[0:5]
```

[109]:
```
array([0.282 , 0.517 , 0.8975, 0.6327, 0.8768])
```

6.0.2 Interpretación de las probabilidades predichas:

El modelo calcula la probabilidad de que cada paciente pertenezca a la clase 1, correspondiente a medicamentos de origen extranjero. Estas probabilidades permiten observar qué tan segura es la predicción realizada por el modelo para cada paciente.

[110]:
```
y_test[0:5]
```

[110]:
```
176    0
41     0
35     1
91     1
146    1
Name: Drug, dtype: int64
```

6.0.3 Comparación con los valores reales

Los valores de y_test representan la clasificación real de los pacientes del conjunto de prueba, donde 0 corresponde a medicamentos nacionales y 1 a medicamentos extranjeros.

Al comparar estos valores con las probabilidades y predicciones obtenidas por el modelo, es posible identificar si la clasificación realizada fue correcta o incorrecta.

[111]:
```
import numpy as np
y_pred_probs = np.round(y_pred_probs,0)
y_pred_probs[0:5]
```

[111]:
```
array([0., 1., 1., 1., 1.])
```

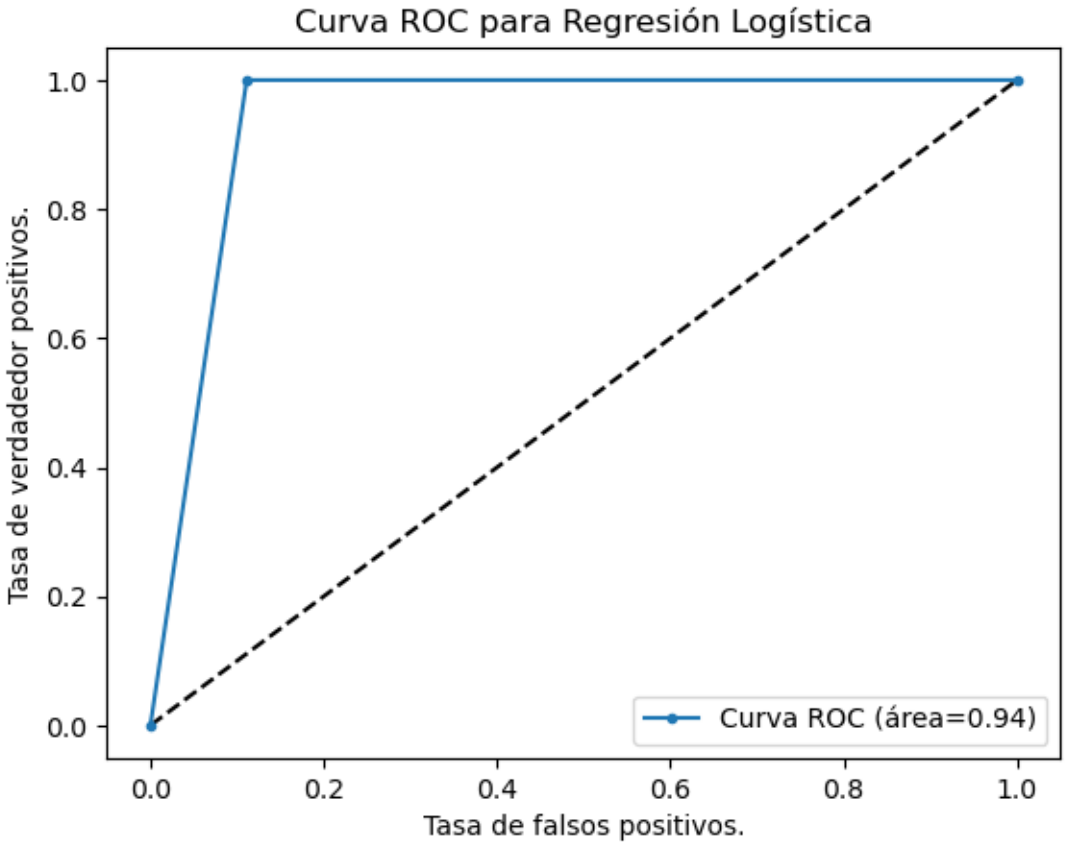
[112]:
```
#Graficación de la curva ROC y cálculo del área bajo ella
from sklearn.metrics import roc_curve
from sklearn.metrics import roc_auc_score

#fpr =false positive rate
#tpr = true positive rate
#threshols = limites para valores de p
```

```
fpr,tpr, thresholds = roc_curve(y_test,y_pred_probs)
auc = roc_auc_score(y_test, y_pred_probs)
```

[113]:
```
plt.plot([0,1],[0,1], 'k--')
plt.plot (fpr,tpr,marker=".",label = "Curva ROC (área=%0.2f)" % auc)
plt.xlabel("Tasa de falsos positivos.")
plt.ylabel("Tasa de verdadedor positivos.")
plt.legend(loc= "lower right")
plt.title("Curva ROC para Regresión Logística")
```

[113]: Text(0.5, 1.0, 'Curva ROC para Regresión Logística')



6.0.4 Interpretación de la curva ROC

La curva ROC permite evaluar qué tan bien el modelo distingue entre medicamentos nacionales y extranjeros. En este caso, la curva se encuentra claramente por encima de la línea diagonal, lo que indica que el modelo clasifica mejor que si lo hiciera al azar.

El área bajo la curva (AUC) es aproximadamente de 0.95. Este valor es cercano a 1, por lo que indica una capacidad muy buena del modelo para diferenciar entre las dos clases.

Además, la curva se aproxima a la esquina superior izquierda, lo cual representa una alta tasa de verdaderos positivos y una baja tasa de falsos positivos.

El modelo seleccionado con Liblinear presenta un buen desempeño para clasificar si un medicamento es de origen nacional o extranjero. Este resultado coincide con el reporte de clasificación, donde Liblinear obtuvo la mayor precisión global entre los solucionadores evaluados, por lo que se considera el método más adecuado para este conjunto de datos.